FUTURE INTERNS · CYBER SECURITY TASK 1

Vulnerability Assessment Report

Read-only passive web application assessment

Target	http://owasp-juice.shop
Assessment date	21 October 2026
Assessment type	Read-only passive web application assessment
Primary tools	OWASP ZAP 2.17.0 · Firefox DevTools
Prepared for	Future Interns — Cyber Security Internship

Assessment boundary: Only public-facing resources were reviewed using passive/read-only techniques. No exploitation, brute force, login bypass, or denial-of-service activity was performed.

FINAL REPORT

01 · Executive Summary

A read-only passive assessment of the OWASP Juice Shop project website (owasp-juice.shop, served from juice-shop.github.io) identified four medium-risk findings, one low-risk finding and one informational finding. The most significant items were the missing Content-Security-Policy header, a Cross-Origin Resource Sharing (CORS) misconfiguration, the missing anti-clickjacking header, and missing Subresource Integrity attributes on externally loaded resources.

Because the assessment was intentionally non-invasive, these results are security observations and remediation priorities—not proof that the application is actively exploitable.

Risk Overview

Severity	Finding	Priority
MEDIUM	Content Security Policy (CSP) header not set	1 — Implement
MEDIUM	Cross-Domain (CORS) misconfiguration	2 — Restrict
MEDIUM	Missing anti-clickjacking header	3 — Implement
MEDIUM	Sub Resource Integrity attribute missing	4 — Implement
LOW	X-Content-Type-Options header missing	5 — Implement
INFO	Information disclosure — suspicious comments	6 — Review

Important: ZAP alerts were not automatically treated as exploitable vulnerabilities.

02 · Scope & Methodology

Target

http://owasp-juice.shop — the public OWASP Juice Shop project website, a static site served from GitHub Pages (juice-shop.github.io). This is the project's information site, not the intentionally vulnerable Juice Shop application.

Assessment objectives

- Review HTTP response headers and security configuration.
- Use OWASP ZAP in passive mode to identify observable web security issues.
- Classify findings by risk level and provide practical remediation.

Allowed / Prohibited Activity

Performed: public-facing page review, passive scanning, header checks, and configuration analysis.

Not performed: exploitation, login bypass, brute force, denial-of-service, or activity intended to harm the target.

Tools

OWASP ZAP 2.17.0 — passive analysis · Firefox DevTools — browser/network inspection

Limitations

ZAP alerts are automated observations and should be validated by the application owner before remediation. Header and configuration findings are hardening observations, not confirmed exploits. The site is hosted on GitHub Pages, so response headers are set by the hosting platform and some fixes may need a proxy or CDN in front of the site. Third-party domains loaded by the page (fonts, CDN, badge and social-embed hosts) appeared in ZAP's site tree because the browser loaded them; they were not assessed.

03 · Finding F-01

F-01 · Content Security Policy (CSP) Header Not Set

MEDIUM

Confidence: High · CWE-693

Summary

The application does not set a Content-Security-Policy (CSP) HTTP response header. CSP tells the browser which sources are allowed to load content such as scripts, styles, images, fonts and frames on a page. It helps reduce the impact of attacks like Cross-Site Scripting (XSS) and data injection.

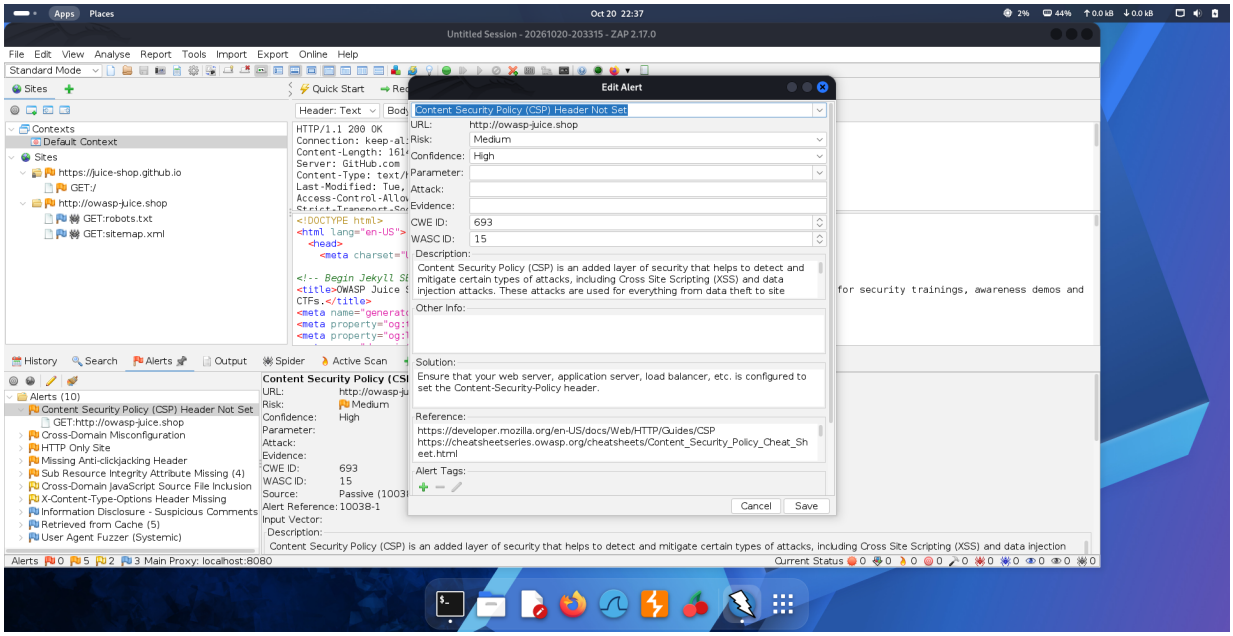
Business impact

Without a CSP, the browser does not restrict where scripts and other content can be loaded from. If an attacker manages to inject malicious content through another vulnerability, the browser has no policy to block it. This can make attacks like XSS more damaging.

Remediation

Ensure that your web server, application server, load balancer, etc. is configured to set the Content-Security-Policy header.

Evidence



OWASP ZAP passive evidence captured during the assessment. Sensitive cookie/session values are not included.

03 · Finding F-02

F-02 · Cross-Domain Misconfiguration MEDIUM

Confidence: Medium · CWE-264

Summary

The web server has a Cross-Origin Resource Sharing (CORS) misconfiguration. The `Access-Control-Allow-Origin` header is set too permissively (ZAP evidence: `Access-Control-Allow-Origin: *`), which may allow web pages from other domains to load data from this application in the browser.

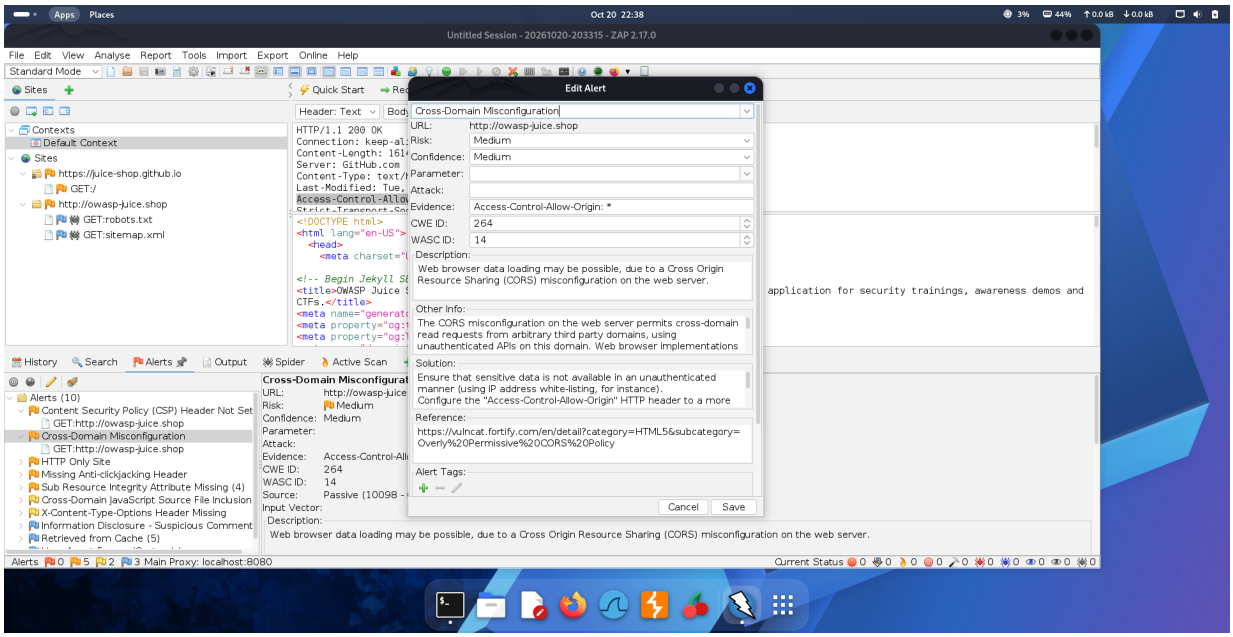
Business impact

If a website allows requests from any origin, a malicious website can make a user's browser read its responses. If any of that data is sensitive and not protected by authentication, it could be exposed to other domains.

Remediation

Make sure sensitive data is not available without authentication (for example, use IP address allow-listing). Set the `Access-Control-Allow-Origin` header to a short list of trusted domains, or remove the CORS headers completely so the browser enforces the Same Origin Policy (SOP).

Evidence



OWASP ZAP passive evidence captured during the assessment. Sensitive cookie/session values are not included.

03 · Finding F-03

F-03 · Missing Anti-clickjacking Header

MEDIUM

Confidence: Medium · CWE-1021

Summary

The application's response does not protect against clickjacking attacks. It does not include a Content-Security-Policy header with the `frame-ancestors` directive, or an `X-Frame-Options` header.

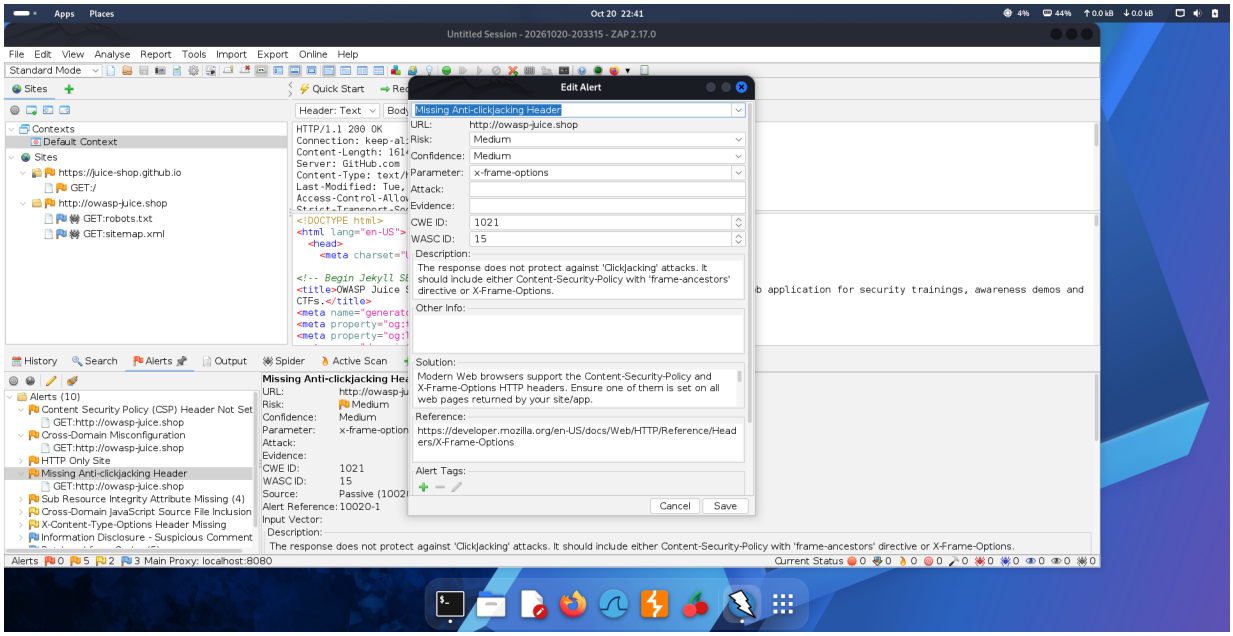
Business impact

Without these headers, another website can load this application's pages inside a hidden or disguised frame. An attacker can then trick a user into clicking something on the page without realizing it, which may cause the user to perform unintended actions.

Remediation

Set either the Content-Security-Policy header (with `frame-ancestors`) or the `X-Frame-Options` header on all pages. Use `DENY` if the page should never be framed, or `SAMEORIGIN` if it should only be framed by pages on the same server.

Evidence



OWASP ZAP passive evidence captured during the assessment. Sensitive cookie/session values are not included.

03 · Finding F-04

F-04 · Sub Resource Integrity Attribute Missing

MEDIUM

Confidence: High · CWE-345

Summary

The `integrity` attribute is missing on a `<script>` or `<link>` tag that loads a file from an external server. This attribute lets the browser check that the file has not been changed.

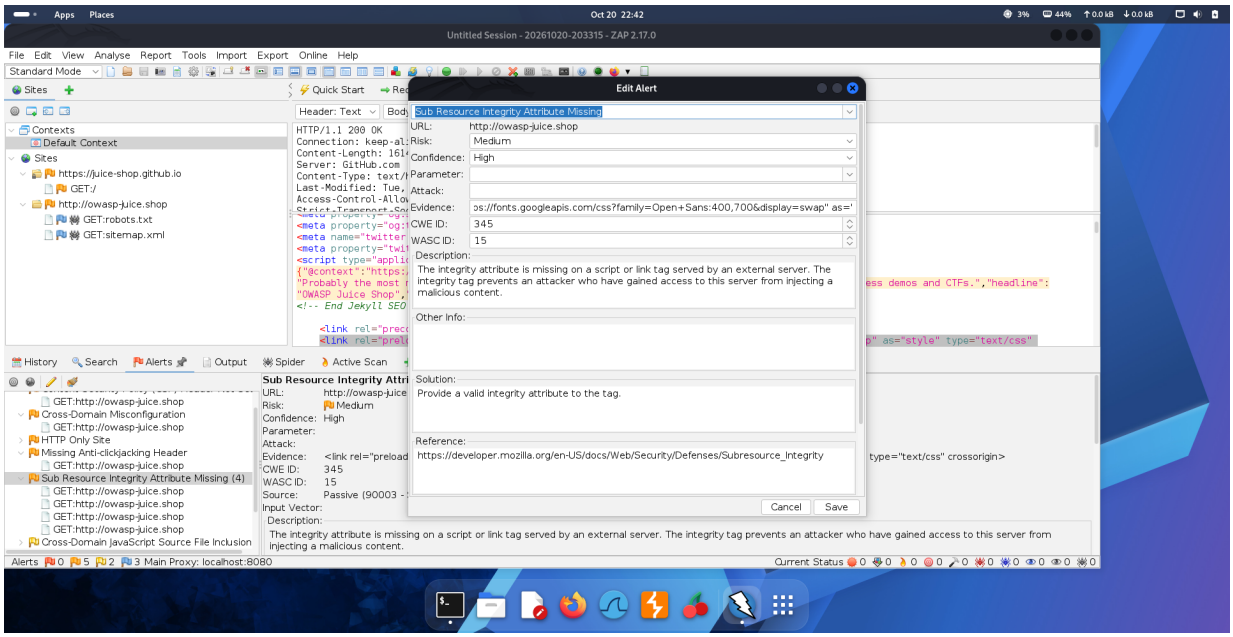
Business impact

If the external server is compromised, an attacker can change the file it serves and inject malicious content. Without the integrity attribute, the browser will load and run the modified file without any check, which could affect everyone who visits the application.

Remediation

Add a valid `integrity` attribute to the `<script>` or `<link>` tag that loads the external resource.

Evidence



OWASP ZAP passive evidence captured during the assessment. Sensitive cookie/session values are not included.

03 · Finding F-05

F-05 · X-Content-Type-Options Header Missing LOW

Confidence: Medium · CWE-693

Summary

The `X-Content-Type-Options` header is not set to `nosniff`. Because of this, some browsers (mainly older versions of Internet Explorer and Chrome) may try to guess the content type of a response instead of using the type declared by the server.

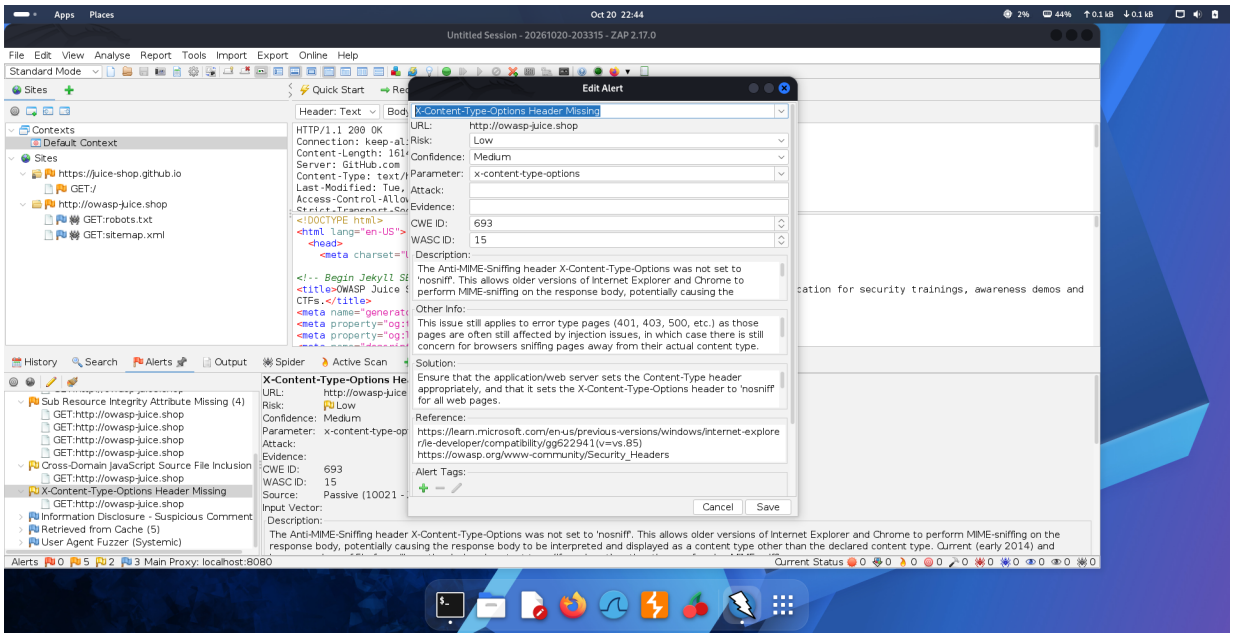
Business impact

If a browser guesses the content type, it may treat a response as a different type than the server intended, for example showing or running content that should have been treated as plain data. This could be abused to deliver unexpected content to users. The risk is low because modern browsers such as Firefox use the declared content type.

Remediation

Make sure the server sets the `Content-Type` header correctly, and set `X-Content-Type-Options: nosniff` on all web pages. If possible, encourage users to use a modern, standards-compliant browser that does not perform MIME-sniffing.

Evidence



OWASP ZAP passive evidence captured during the assessment. Sensitive cookie/session values are not included.

03 · Finding F-06

F-06 · Information Disclosure - Suspicious Comments

INFO

Confidence: Medium · CWE-615

Summary

The response contains comments that may reveal information about the application, such as unfinished work or how it is built. These comments are visible to anyone who views the page source.

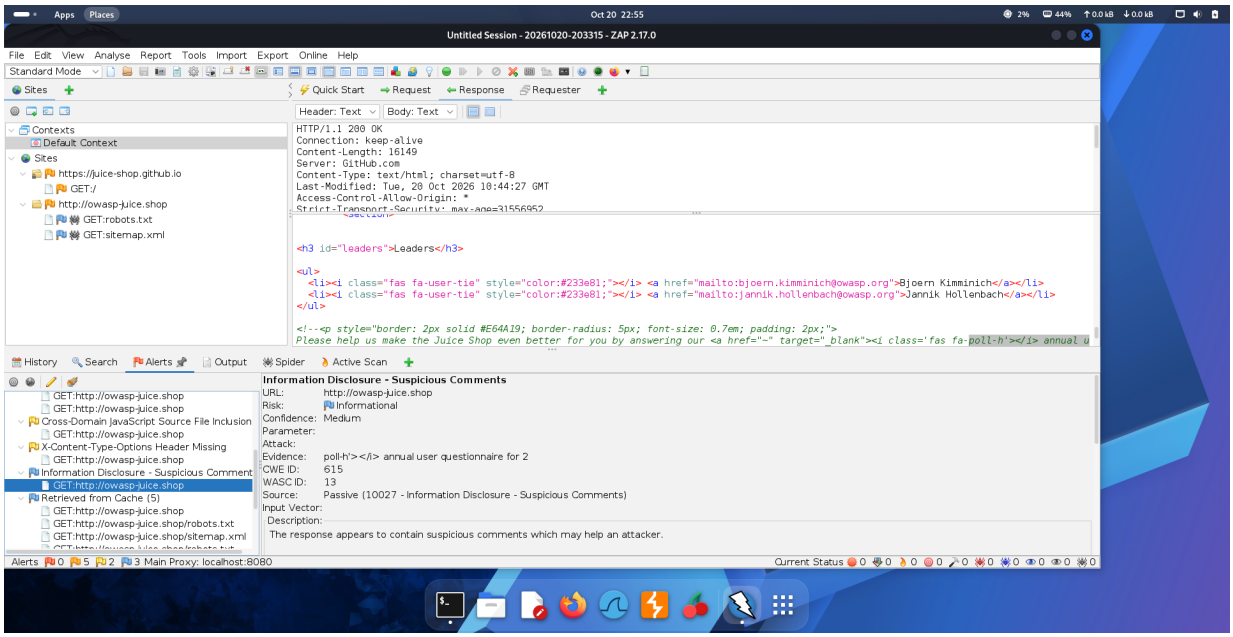
Business impact

On its own, this is not a vulnerability. However, comments left in public code may give an attacker small hints about the application's internals or known problems, which could help them plan an attack.

Remediation

Remove all comments that reveal information an attacker could use, and fix any underlying problems those comments refer to.

Evidence

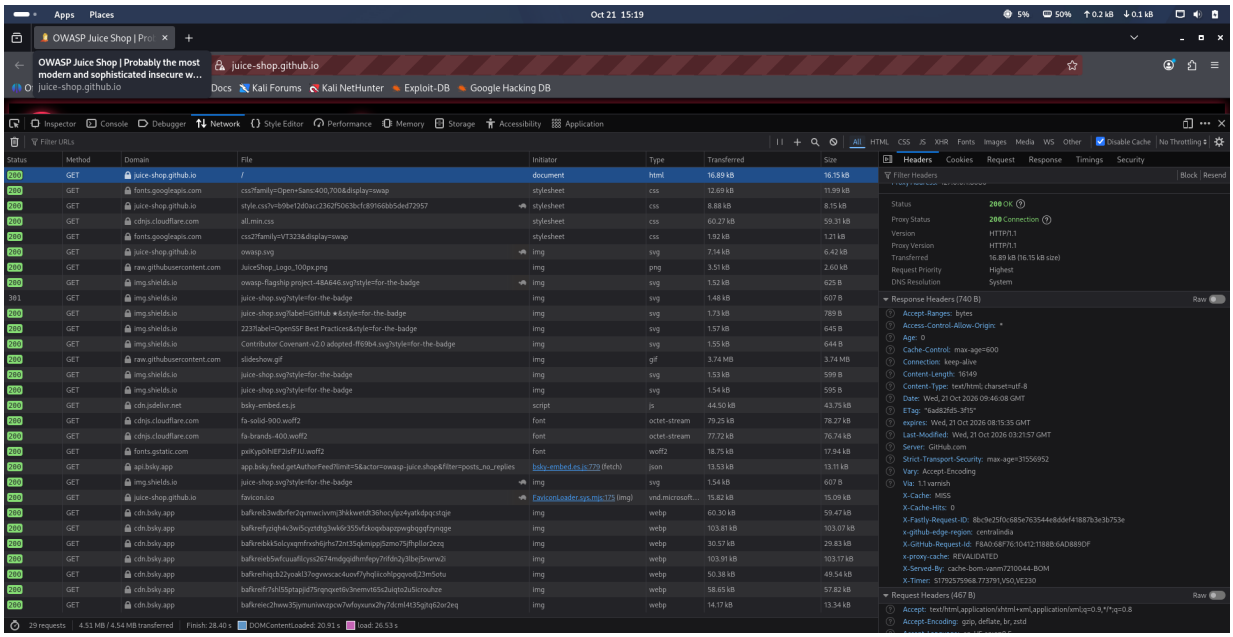


OWASP ZAP passive evidence captured during the assessment. Sensitive cookie/session values are not included.

05 · Browser Header Verification

The ZAP results were cross-checked manually in Firefox Developer Tools (Network tab, HTTP 200 response with the browser cache disabled). The request passed through the local ZAP proxy.

- `Access-Control-Allow-Origin: *` and `Strict-Transport-Security: max-age=31556952` were present in the response, consistent with finding F-02 and showing that HSTS is already set.
- No `Content-Security-Policy`, `X-Frame-Options` or `X-Content-Type-Options` header appeared in the response headers, consistent with findings F-01, F-03 and F-05.
- The `Server: GitHub.com` and `Via: 1.1 varnish` headers confirm the site is served by GitHub Pages behind a caching layer.



Firefox Developer Tools — response headers for the main page. No cookie values appear in this capture.

06 · Remediation Roadmap

Priority	Action	Suggested owner
1	Define and deploy a suitable Content-Security-Policy.	Development / Platform
2	Restrict CORS to trusted domains or remove the CORS headers.	Development / Platform
3	Set X-Frame-Options or CSP frame-ancestors on all pages.	Platform / DevOps
4	Add integrity attributes to externally loaded scripts and styles.	Development
5	Enable X-Content-Type-Options: nosniff.	Platform / DevOps
6	Remove revealing comments from public code.	Development
7	Keep dependencies and security headers under continuous review.	Security / Engineering

Retest recommendation: after remediation, rerun the passive ZAP scan and verify response headers using browser Developer Tools or another safe HTTP header check. Confirm that the missing headers are present and that externally loaded resources carry integrity attributes. Because the site is served by GitHub Pages, header changes may need a proxy or CDN in front of the site.

Recommended GitHub Structure

```
FUTURE_CS_01/
+-- Nmap/
+-- Report/
|   +-- FUTURE_CS_01_Vulnerability_Assessment_Report.pdf
+-- owasp_zap/
|   +-- Screenshot/
|   +-- zap-findings.md
+-- README.md
+-- assessment_info.md
```

Publishing rule: remove or redact cookie/session values, tokens, authorization headers, or other secrets from screenshots and logs before uploading evidence to GitHub.

07 · Conclusion

This assessment provides a prioritized, business-friendly view of the security weaknesses observable within a read-only scope. Browser security-header hardening and third-party resource integrity are the top priorities. The findings should be retested after remediation. No claim of exploitability is made without an authorized validation exercise.

Assessment status	**FINAL — READY FOR SUBMISSION**
Target	http://owasp-juice.shop
Method	Read-only passive assessment
Evidence	OWASP ZAP + browser/network inspection
Sensitive data handling	Cookie/session values excluded from published evidence

Prepared for Future Interns — Cyber Security Internship · Task 1